

SCX

State-Conditioned eXpertise — : v0.1.0 | : 2026-06-25

1. Python
 - 2.
 - 3.
 4. ML
 5. MLIP
 - 6.
-

1. Python

1.1

```
src/scx/
__init__.py          # API
core/
  __init__.py
  framework.py       # SCXFramework -
  config.py          # SCXConfig -
  metrics.py         # SCXMetrics -
state/
  __init__.py
  space.py           # StateSpace -
  discovery.py       # StateDiscovery -
  assignment.py      # StateAssignment - /
  metrics.py         # StateMetrics -
expert/
  __init__.py
  registry.py        # ExpertRegistry -
  reliability.py     # ExpertReliability -
  router.py          # ExpertRouter -
```

```

    conflict.py                # ExpertConflict -
valuation/
    __init__.py
    classifier.py              # DataClassifier -
    state_value.py             # StateValue - V(s)
    learnability.py            # LearnabilityScore - L(s)
    noise_score.py             # NoiseScore - N(x)
    redundancy.py              # RedundancyScore - D(s)
action/
    __init__.py
    policy.py                  # ActionPolicy -
    acquisition.py             # AcquisitionStrategy -
    compress.py                # CompressStrategy -
utils/
    __init__.py
    visualization.py           # (plotly + matplotlib)
    data_loader.py             #
    helpers.py                 #
    evaluation.py              #

```

1.2

scx/state/ — scx/state/space.py — StateSpace

SCX

```

@dataclass
class StateInfo:
    """
    id: int                    #
    label: str                 #
    centroid: np.ndarray       #
    radius: float              #
    count: int                 #
    proportion: float          # (s) = count / total

class StateSpace:

```

```

""" S = {s_1, ..., s_K}"""

def __init__(self, n_states: int):
    self.n_states = n_states
    self.states: dict[int, StateInfo] = {}
    self.assignment: np.ndarray | None = None      # (N,)
    self.soft_assignment: np.ndarray | None = None # (N, K)

    def add_state(self, state: StateInfo) -> None
    def get_state(self, state_id: int) -> StateInfo
    def remove_state(self, state_id: int) -> None
    def get_centroids(self) -> np.ndarray          # shape (K, d)
    def get_proportions(self) -> np.ndarray         # shape (K,)
    def summary(self) -> pd.DataFrame              #
    def save(self, path: str) -> None              #
    @classmethod
    def load(cls, path: str) -> StateSpace

```

numpy, pandas, pickle

scx/state/discovery.py — StateDiscovery

(X)

```

class StateDiscovery:
    """ (X) /+"""

    def __init__(self, method: str = "kmeans", n_states: int = 10,
                  random_state: int = 42):
        """
        Parameters
        -----
        method : str
            'kmeans' | 'gmm' | 'spectral' | 'hdbscan' | 'vae'
        n_states : int
            HDBSCAN
        """
        self.method = method

```

```

        self.n_states = n_states
        self.model = None

    def fit(self, X_phi: np.ndarray) -> "StateDiscovery":
        """ (X)

        Parameters
        -----
        X_phi : np.ndarray, shape (N, d)

        """
        ...

    def predict(self, X_phi: np.ndarray) -> np.ndarray:
        """ (x) ID"""
        ...

    def fit_predict(self, X_phi: np.ndarray) -> np.ndarray:
        """
        ...

    def get_centroids(self) -> np.ndarray:
        """ shape (K, d)"""
        ...

    def get_probabilities(self, X_phi: np.ndarray) -> np.ndarray:
        """ shape (N, K)
           one-hot
        """
        ...

```

sklearn.cluster, sklearn.mixture, hdbscan

scx/state/assignment.py — StateAssignment

```

class StateAssignment:
    """/"""

```

```

@staticmethod
def hard_assign(
    X_phi: np.ndarray,
    centroids: np.ndarray,
    metric: str = "euclidean"
) -> np.ndarray:
    """

    Returns: (N,) int
    """
    ...

@staticmethod
def soft_assign(
    X_phi: np.ndarray,
    centroids: np.ndarray,
    temperature: float = 1.0,
    metric: str = "euclidean"
) -> np.ndarray:
    """ softmax

    Returns: (N, K) float , 1
    """
    ...

@staticmethod
def soft_assign_gmm(
    X_phi: np.ndarray,
    means: np.ndarray,
    covariances: np.ndarray,
    weights: np.ndarray
) -> np.ndarray:
    """GMM """
    ...

@staticmethod
def state_proportions(assignment: np.ndarray, n_states: int) -> np.ndarray:
    """ (s) """

```

...
numpy, scipy.spatial.distance

scx/state/metrics.py — StateMetrics

```
class StateMetrics:
    """

    @staticmethod
    def silhouette_score(X_phi: np.ndarray, labels: np.ndarray) -> float:
        """

        ...

    @staticmethod
    def calinski_harabasz(X_phi: np.ndarray, labels: np.ndarray) -> float:
        """CH /"""

        ...

    @staticmethod
    def davies_bouldin(X_phi: np.ndarray, labels: np.ndarray) -> float:
        """DB /"""

        ...

    @staticmethod
    def state_purity(
        labels_pred: np.ndarray,
        labels_true: np.ndarray
    ) -> float:
        """

        ...

    @staticmethod
    def state_cohesion(X_phi: np.ndarray, labels: np.ndarray) -> dict:
        """

        ...

    @staticmethod
```

```

def stability_analysis(
    X_phi: np.ndarray,
    method: str = "kmeans",
    n_runs: int = 20,
    k_range: list[int] = None
) -> dict:
    """ k """
    ...

```

sklearn.metrics

scx/expert/ — scx/expert/registry.py — ExpertRegistry

```

class ExpertInfo(NamedTuple):
    name: str #
    predict_fn: Callable # fm: X -> Y
    metadata: dict #

class ExpertRegistry:
    """

    def __init__(self):
        self._experts: dict[str, ExpertInfo] = {}

    def register(
        self, name: str,
        predict_fn: Callable,
        metadata: dict | None = None
    ) -> None:
        """
        ...

    def unregister(self, name: str) -> None:
        """
        ...

    def get_expert(self, name: str) -> ExpertInfo:

```



```

        """
        ...

    def list_experts(self) -> list[str]:
        """
        ...

    def get_predict_fns(self) -> list[Callable]:
        """
        ...

    def evaluate_all(
        self, X: np.ndarray, y: np.ndarray,
        metric: Callable = accuracy_score
    ) -> dict[str, float]:
        """
        ...

    @property
    def size(self) -> int:
        ...

```

typing, Callable

scx/expert/reliability.py — ExpertReliability

$R_m(s)$ $SCX_m(s)$

```

@dataclass
class ReliabilityResult:
    """
    risk_matrix: np.ndarray          # shape (M, K) -  $R_m(s)$ 
    scx_matrix: np.ndarray           # shape (M, K) -  $SCX_m(s)$ 
    risk_std: np.ndarray             # shape (M, K) -
    n_samples_per_state: np.ndarray # shape (K,) -

class ExpertReliability:
    """

```

```

def __init__(
    self,
    experts: ExpertRegistry,
    loss_fn: Callable = None,
    tau: float = 0.1,
    min_samples: int = 5
):
    """
    Parameters
    -----
    experts : ExpertRegistry
    loss_fn : Callable(y_pred, y_true) -> float
        MSE for regression, 0-1 loss for classification
    tau : float
        SCX
    min_samples : int

    """
    self.experts = experts
    self.loss_fn = loss_fn or self._default_loss_fn
    self.tau = tau
    self.min_samples = min_samples

def estimate_supervised(
    self,
    X: np.ndarray,
    y: np.ndarray,
    state_assignment: np.ndarray,
    n_states: int
) -> ReliabilityResult:
    """
    
$$R_m(s) = (1/|L_s|) * \sum_{(x,y) \in L_s} (f_m(x), y)$$

    
$$SCX_m(s) = (1/|L_s|) * \sum_{(x,y) \in L_s} 1[(f_m(x), y) < ]$$

    """
    ...

```

```

def estimate_unsupervised(
    self,
    X: np.ndarray,
    state_assignment: np.ndarray,
    n_states: int,
    n_neighbors: int = 5
) -> np.ndarray:
    """
    /
    1.  $D(s) \sim \{M \times M\}$ 
    2.
    3.

    Returns:  $(M, K)$ 
    """
    ...

def estimate_joint(
    self,
    X_labeled: np.ndarray, y_labeled: np.ndarray,
    X_unlabeled: np.ndarray,
    state_assignment_labeled: np.ndarray,
    state_assignment_unlabeled: np.ndarray,
    n_states: int
) -> ReliabilityResult:
    """ + """
    ...

def compute_scx_from_risk(
    self, risk_matrix: np.ndarray
) -> np.ndarray:
    """ SCX

     $SCX_m(s) = \exp(- * R_m(s)) \quad (1 - R_m(s))$ 
    """
    ...

```

```

@staticmethod
def _default_loss_fn(y_pred: np.ndarray, y_true: np.ndarray) -> np.ndarray:
    """
    ...

```

numpy, scipy.spatial.distance, sklearn.metrics

scx/expert/router.py — ExpertRouter

```

class ExpertRouter:
    """

    def __init__(
        self,
        experts: ExpertRegistry,
        reliability: ExpertReliability,
        alpha: float = 1.0,          # softmax
        tau_hard: float = 0.7       #
    ):
        ...
    def hard_route(
        self,
        x: np.ndarray,
        state_weights: np.ndarray,   # (K,)
        risk_matrix: np.ndarray      # (M, K)   $R_m(s)$ 
    ) -> tuple[int, str]:
        """

         $m^*(x) = \arg \min_m \sum_s(x) R_m(s)$ 
        """
        ...

    def soft_weights(
        self,
        x: np.ndarray,
        state_weights: np.ndarray,   # (K,)
        risk_matrix: np.ndarray      # (M, K)
    ) -> np.ndarray:

```

```

        """

         $w_m(x) = \exp(-\sum_s \_s(x) R_m(s))$ 
        """
        ...

    def predict(
        self,
        x: np.ndarray,
        state_weights: np.ndarray,
        risk_matrix: np.ndarray,
        mode: str = "soft"                # "soft" / "hard"
    ) -> np.ndarray:
        """

        soft mode:  $\hat{y} = \sum_m w_m(x) f_m(x)$ 
        hard mode:  $\hat{y} = f_{\{m^*\}}(x)$ 
        """
        ...

```

numpy

scx/expert/conflict.py — ExpertConflict

```

@dataclass
class ConflictResult:
    state_id: int
    conflict_score: float                # 0~1,
    disagreeing_experts: list[str]      #
    consensus_prediction: np.ndarray    #
    conflict_type: str                  # "hard" / "disagreement" / "divergence"

class ExpertConflict:
    """

    @staticmethod
    def pairwise_disagreement(
        predictions: np.ndarray          # (M, N)

```

```

) -> np.ndarray:
    """  $D \in \{M \times M\}$  """
    ...

@staticmethod
def state_conflict_score(
    predictions: np.ndarray,          # (M, N_s)
    method: str = "mad"               # "mad" / "pairwise" / "entropy"
) -> float:
    """
    ...

@staticmethod
def arbitrate(
    predictions: np.ndarray,          # (M,)
    risk_s: np.ndarray,               # (M,)
    method: str = "minimum_risk"
) -> tuple[np.ndarray, str]:
    """
    method:
        "minimum_risk" -  $R_m(s)$ 
        "weighted_vote" -  $\exp(-R_m(s))$ 
        "oracle" - oracle
    """
    ...

@staticmethod
def detect_conflict_states(
    predictions: np.ndarray,          # (M, N)
    state_assignment: np.ndarray,     # (N,)
    n_states: int,
    threshold: float = 0.3
) -> list[ConflictResult]:
    """
    ...

```

numpy, scipy.stats

scx/valuation/ — scx/valuation/state_value.py — StateValue

$V(s)$

```
class StateValue:
    """  $V(s)$ 

    
$$V(s) = r(s) + \gamma \sum_{s'} P(s'|s) L(s') [1 - D(s')] \max_m SCX_m(s)$$

    """

    def __init__(self):
        self.components: dict[str, np.ndarray] = {}

    def compute(
        self,
        mean_error: np.ndarray,          # (K,)  $r(s)$ 
        proportion: np.ndarray,          # (K,)  $\gamma(s)$ 
        learnability: np.ndarray,        # (K,)  $L(s)$ 
        redundancy: np.ndarray,          # (K,)  $D(s)$ 
        scx_reliability: np.ndarray      # (M, K)  $SCX_m(s)$ 
    ) -> np.ndarray:
        """  $V(s)$ 

        Returns: (K,)
        """
        max_scx = np.max(scx_reliability, axis=0) # (K,)
        V = mean_error * proportion * learnability * (1 - redundancy) * max_scx
        self.components = {
            "mean_error": mean_error,
            "proportion": proportion,
            "learnability": learnability,
            "redundancy": redundancy,
            "max_scx": max_scx,
            "value": V
        }
    return V
```

```

def get_components(self) -> dict[str, np.ndarray]:
    """  $V(s)$  """
    ...

def rank_states(self, V: np.ndarray) -> np.ndarray:
    """ """
    ...

def top_k_states(self, V: np.ndarray, k: int) -> np.ndarray:
    """  $k$   $ID$  """
    ...

```

numpy

scx/valuation/classifier.py — DataClassifier

Valuable / Redundant / Noisy / ExpertDependent

```

@dataclass
class ClassificationResult:
    """ """
    state_id: int
    category: str # "valuable" / "redundant" / "noisy" / "expertDependent"
    confidence: float
    reason: str #

class DataClassifier:
    """ """

    def __init__(
        self,
        tau_r: float = 0.3, #
        tau_rho: float = 0.05, #
        tau_L: float = 0.4, #
        tau_D: float = 0.7, #
        tau_conflict: float = 0.3 #
    ):

```



```

...

def classify_state(
    self,
    mean_error: float,          #  $r(s)$ 
    proportion: float,          #  $(s)$ 
    learnability: float,        #  $L(s)$ 
    redundancy: float,          #  $D(s)$ 
    scx_reliability: np.ndarray, #  $(M,)$  SCX
    state_id: int = -1
) -> ClassificationResult:
    """

    max_scx = np.max(scx_reliability)
    min_scx = np.min(scx_reliability)
    scx_range = max_scx - min_scx

    # Valuable: + + +
    if (mean_error > self.tau_r and proportion > self.tau_rho
        and learnability > self.tau_L and redundancy < self.tau_D):
        return ClassificationResult(state_id, "valuable", 0.9,
            f"r={mean_error:.3f}>_r, ={proportion:.3f}>_, L={learnability:.3f}

    # Redundant: + +
    if (mean_error < self.tau_r / 2 and proportion > self.tau_rho
        and redundancy > self.tau_D):
        return ClassificationResult(state_id, "redundant", 0.85,
            f"r={mean_error:.3f}<_r/2, D={redundancy:.3f}>_D")

    # ExpertDependent:
    if scx_range > self.tau_conflict:
        return ClassificationResult(state_id, "expert_dependent", min(0.9, scx_range / self.tau_conflict),
            f"SCX range={scx_range:.3f}>_conflict")

    # Noisy: + +
    if (mean_error > self.tau_r and proportion < self.tau_rho
        and learnability < self.tau_L):
        return ClassificationResult(state_id, "noisy", 0.8,

```

```

        f"r={mean_error:.3f}>_r, ={proportion:.3f}<_, L={learnability:.3f}

    # valuable
    return ClassificationResult(state_id, "valuable", 0.5, "default: conservat

def classify_all_states(
    self,
    mean_errors: np.ndarray,
    proportions: np.ndarray,
    learnabilities: np.ndarray,
    redundancies: np.ndarray,
    scx_reliability: np.ndarray
) -> list[ClassificationResult]:
    """
    ...

def summary(self, results: list[ClassificationResult]) -> dict:
    """
    ...

def classify_sample(
    self,
    sample_error: float,          #  $r(x)$ 
    sample_density: float,        #  $(s(x))$ 
    sample_consistency: float,    #
) -> str:
    """
    ...

```

numpy, dataclasses

scx/valuation/learnability.py — LearnabilityScore

$L(s)$

```

class LearnabilityScore:
    """  $L(s)$ 

```

```


$$L(s) = \frac{C(s)}{C(s) + N(s)} [1 - N(s)]$$

"""

@staticmethod
def from_label_consistency(
    y: np.ndarray, #
    method: str = "entropy"
) -> float:
    """
    
$$C(s) = 1 - H(Y/s) / \log(|Y|)$$

    """
    ...

@staticmethod
def from_expert_consensus(
    predictions: np.ndarray, # (M, N_s)
    labels: np.ndarray # (N_s,)
) -> float:
    """
    ...

@staticmethod
def from_knn_consistency(
    X_phi: np.ndarray, # (N_s, d)
    labels: np.ndarray, # (N_s,)
    n_neighbors: int = 5
) -> float:
    """ k-NN
    →
    → /
    """
    ...

@staticmethod
def estimate_noise_fraction(

```

```

        X_phi: np.ndarray,
        y: np.ndarray,
        method: str = "knn"
    ) -> float:
        """  $N(s)$  """
        ...

```

numpy, scipy.stats, sklearn.neighbors

scx/valuation/noise_score.py — NoiseScore

$N(x)$

```

class NoiseScore:
    """

```

```

        NoiseScore( $x_i$ ) =  $r_i \cdot (1/(s_i) + ) \cdot [1 - C(s_i)]$ 
    """

```

```

    @staticmethod

```

```

    def estimate(
        errors: np.ndarray,          # (N,)  $r_i$  -
        state_densities: np.ndarray, # (N,)  $(s_i)$  -
        consistencies: np.ndarray,   # (N,)  $C(s_i)$  -
        epsilon: float = 1e-8
    ) -> np.ndarray:
        """

```

```

            + + +
            + + +
        """
        ...

```

```

    @staticmethod

```

```

    def detect_label_noise(
        X_phi: np.ndarray,
        y: np.ndarray,
        n_neighbors: int = 5,

```

```

        threshold: float = 0.5
    ) -> np.ndarray:
        """

        : k-NN
        Returns: (N,) bool mask - True
        """

        ...

    @staticmethod
    def detect_outlier_noise(
        X_phi: np.ndarray,
        centroids: np.ndarray,
        radii: np.ndarray,
        assignment: np.ndarray,
        n_std: float = 3.0
    ) -> np.ndarray:
        """

        ...

    @staticmethod
    def rank_by_noise(
        X_phi: np.ndarray,
        y: np.ndarray,
        errors: np.ndarray,
        state_info: StateSpace
    ) -> np.ndarray:
        """

        ...

```

numpy, sklearn.neighbors, scipy.spatial

scx/valuation/redundancy.py — RedundancyScore

$D(s)$

```

class RedundancyScore:
    """  $D(s)$ 

```

```

D(s) s :
- D(s) →
- D(s) →
"""

@staticmethod
def from_coverage(
    X_phi: np.ndarray,          # (N_s, d)
    centroid: np.ndarray,      # (d,)
    radius: float
) -> float:
    """

    D(s) = min(1, coverage_volume / total_volume)
    """
    ...

@staticmethod
def from_nearest_neighbor_distance(
    X_phi: np.ndarray,
    n_neighbors: int = 3
) -> float:
    """

    → →
    D(s) = 1 - tanh(mean_knn_dist / scale)
    """
    ...

@staticmethod
def from_information_gain(
    X_phi: np.ndarray,
    y: np.ndarray,
    model: Callable = None
) -> float:
    """

```

```

        →
        """
        ...

    @staticmethod
    def from_diversity(
        X_phi: np.ndarray,
        method: str = "determinantal_point_process"
    ) -> float:
        """

         $D(s) = 1 - \text{diversity}(s)$ 
        DPP
        """
        ...

```

numpy, sklearn.neighbors, scipy.linalg

scx/action/ — scx/action/policy.py — ActionPolicy

```

class ActionPolicy:
    """

     $a^*(s) = \arg \max_a U(a, s)$ 

    : {acquire, relabel, downweight, discard, route}
    """

    def __init__(self, config: dict = None):
        self.config = config or {}

    def decide(
        self,
        classification: ClassificationResult,
        state_value: float,
        budget_remaining: float,

```

```

        context: dict = None
    ) -> str:
        """
        action_map = {
            "valuable": "acquire",
            "redundant": "skip",
            "noisy": "downweight",
            "expert_dependent": "route"
        }
        base_action = action_map.get(classification.category, "acquire")
        # budget
        return self._refine_action(base_action, state_value, budget_remaining, context)

    def decide_batch(
        self,
        classifications: list[ClassificationResult],
        state_values: np.ndarray,
        budget: float,
        context: dict = None
    ) -> dict[int, str]:
        """
        budget
        Returns: {state_id: action}
        """
        ...

    def _refine_action(
        self, action: str,
        state_value: float,
        budget_remaining: float,
        context: dict
    ) -> str:
        """
        ...

```

dataclasses, numpy

scx/action/acquisition.py — AcquisitionStrategy

```
class AcquisitionStrategy:
    """
    point-wise uncertainty sampling
    """

    @staticmethod
    def proportional_allocation(
        state_values: np.ndarray,      # (K,) V(s)
        budget: int
    ) -> np.ndarray:
        """
         $B_s = B * V(s) / V(s')$ 
        Returns: (K,)
        """
        ...

    @staticmethod
    def threshold_allocation(
        state_values: np.ndarray,
        budget: int,
        percentile: float = 70.0
    ) -> np.ndarray:
        """ top """
        ...

    @staticmethod
    def hybrid_allocation(
        state_values: np.ndarray,
        budget: int,
        percentile: float = 60.0,
        min_alloc: int = 1
    ) -> np.ndarray:
        """ """
```

```

...

@staticmethod
def select_samples_in_state(
    X_phi: np.ndarray,          # (N_s, d)
    X_original: np.ndarray,     # (N_s, d_x)
    n_select: int,
    strategy: str = "diverse"   # "random" / "uncertainty" / "diverse" / "coreset"
) -> np.ndarray:
    """ n_select

    "diverse" - k-center
    "uncertainty" -
    "coreset" -
    """
    ...

@staticmethod
def state_uncertainty_score(
    X_phi: np.ndarray,
    model_probs: np.ndarray,
    method: str = "margin"
) -> np.ndarray:
    """
    ...

```

numpy, sklearn.metrics.pairwise

scx/action/compress.py — CompressStrategy

coreset

```

class CompressStrategy:
    """

    """

```

```

@staticmethod
def weighted_random(
    X_phi: np.ndarray,
    y: np.ndarray,
    weights: np.ndarray,
    keep_ratio: float
) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
    """
    ...

@staticmethod
def kcenter_greedy(
    X_phi: np.ndarray,
    y: np.ndarray,
    weights: np.ndarray,
    n_keep: int
) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
    """k-center """
    ...

@staticmethod
def herding(
    X_phi: np.ndarray,
    y: np.ndarray,
    n_keep: int
) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
    """Herding """
    ...

@staticmethod
def weighted_coreset(
    X_phi: np.ndarray,
    y: np.ndarray,
    redundancy_scores: np.ndarray,
    keep_ratio: float = 0.5,
    method: str = "weighted_random"
) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
    """SCX-Compress

```

```

1.  $w_i = \text{softmax}(-D_i)$ 
2.  $\text{keep\_ratio}$   $n_{\text{keep}}$ 
3.  $(X_{\text{selected}}, y_{\text{selected}}, \text{sample\_weights})$ 

```

Returns

```

X_sub : np.ndarray
y_sub : np.ndarray
weights : np.ndarray
"""

```

...

numpy, sklearn.metrics.pairwise

scx/core/ — `scx/core/framework.py` — SCXFramework

SCX

```

class SCXFramework:

```

```

    """SCX

```

```

    """

```

```

def __init__(self, config: "SCXConfig"):
    self.config = config

```

```

    # fit/run

```

```

self.state_discovery: StateDiscovery | None = None
self.state_space: StateSpace | None = None
self.expert_registry: ExpertRegistry | None = None
self.expert_reliability: ExpertReliability | None = None
self.expert_router: ExpertRouter | None = None
self.data_classifier: DataClassifier | None = None
self.state_value: StateValue | None = None
self.action_policy: ActionPolicy | None = None

```

```

#
self.history: list[dict] = []
self.step_count: int = 0

def fit(
    self,
    X_labeled: np.ndarray,
    y_labeled: np.ndarray,
    X_unlabeled: np.ndarray | None = None,
    experts: ExpertRegistry | None = None,
    phi_fn: Callable | None = None
) -> "SCXFramework":
    """ SCX

    Pipeline:
    1. (X)
    2. StateDiscovery.fit(X_phi)
    3. StateSpace ←
    4. ExpertReliability.estimate_joint(L, U)
    5. L(s), D(s), V(s),
    """
    ...

def step(
    self,
    budget: int = 10,
    context: dict = None
) -> dict:
    """ SCX

    1. →
    2. ///
    3.
    4.
    5.

    Returns: dict

```

```

        """
        ...

def run_pipeline(
    self,
    X_labeled: np.ndarray,
    y_labeled: np.ndarray,
    X_unlabeled: np.ndarray,
    experts: ExpertRegistry,
    n_steps: int = 10,
    budget_per_step: int = 10,
    phi_fn: Callable = None
) -> dict:
    """ SCX

    Returns:
    """
    ...

def evaluate(self, X_test: np.ndarray, y_test: np.ndarray) -> dict:
    """
    ...

def get_report(self) -> dict:
    """
    ...

def save(self, path: str) -> None:
    """
    ...

@classmethod
def load(cls, path: str) -> "SCXFramework":
    """
    ...

```

```

@dataclass
class SCXConfig:
    """SCX

    yaml/json
    """
    #
    n_states: int = 10
    state_discovery_method: str = "kmeans"
    state_discovery_kwargs: dict = field(default_factory=dict)

    #
    phi_method: str = "identity"    # "identity" / "pca" / "autoencoder"
    phi_kwargs: dict = field(default_factory=dict)

    #
    tau: float = 0.1                # SCX
    min_samples_per_state: int = 5
    loss_function: str = "auto"     # "auto" / "mse" / "01"

    #
    tau_r: float = 0.3              #
    tau_rho: float = 0.05           #
    tau_L: float = 0.4              #
    tau_D: float = 0.7              #
    tau_conflict: float = 0.3       #

    #
    default_action: str = "acquire"
    acquisition_strategy: str = "hybrid"    # "proportional" / "threshold" / "hybr
    compress_method: str = "weighted_random"
    compress_keep_ratio: float = 0.5

    #
    routing_mode: str = "soft"       # "soft" / "hard"
    routing_alpha: float = 1.0
    routing_tau_hard: float = 0.7

```

```

def validate(self) -> bool:
    """
    ...

def to_dict(self) -> dict:
    ...

@classmethod
def from_dict(cls, d: dict) -> "SCXConfig":
    ...

@classmethod
def from_yaml(cls, path: str) -> "SCXConfig":
    ...

def save(self, path: str) -> None:
    ...

```

dataclasses, pyyaml

scx/core/metrics.py — SCXMetrics

SCX

```

class SCXMetrics:
    """SCX """

    @staticmethod
    def routing_accuracy(
        X: np.ndarray, y: np.ndarray,
        state_space: StateSpace,
        experts: ExpertRegistry,
        risk_matrix: np.ndarray
    ) -> float:
        """
        ...

    @staticmethod

```



```

def valuation_correlation(
    V_estimated: np.ndarray,
    V_true: np.ndarray
) -> dict:
    """
    Returns: {"spearman": , "pearson": r, "kendall": }
    """
    ...

@staticmethod
def acquisition_efficiency(
    history: list[dict],
    metric: str = "auc"
) -> float:
    """budget vs """
    ...

@staticmethod
def compression_retention(
    original_perf: float,
    compressed_perf: float
) -> float:
    """/"""
    ...

@staticmethod
def classification_confusion(
    true_categories: np.ndarray,
    predicted_categories: np.ndarray
) -> dict:
    """
    """
    ...

@staticmethod
def state_valuation_improvement(
    before: dict, after: dict
) -> dict:

```

```

        """ SCX """
        ...
numpy, scipy.stats, sklearn.metrics

```

scx/utils/ — scx/utils/visualization.py

```

class SCXVisualizer:
    """SCX """

    @staticmethod
    def plot_state_space(
        X_phi: np.ndarray,
        assignment: np.ndarray,
        centroids: np.ndarray,
        save_path: str = None
    ) -> None:
        """ 2D/3D PCA/t-SNE """
        ...

    @staticmethod
    def plot_reliability_matrix(
        risk_matrix: np.ndarray,
        expert_names: list[str],
        state_labels: list[str],
        save_path: str = None
    ) -> None:
        """  $R_m(s)$   $SCX_m(s)$  """
        ...

    @staticmethod
    def plot_state_values(
        state_values: np.ndarray,
        components: dict[str, np.ndarray],
        save_path: str = None
    ) -> None:
        """  $V(s)$  """

```

```

...

@staticmethod
def plot_decision_boundary(
    X: np.ndarray, y: np.ndarray,
    states: np.ndarray,
    save_path: str = None
) -> None:
    """
    ...

@staticmethod
def plot_acquisition_curve(
    history: list[dict],
    save_path: str = None
) -> None:
    """budget vs """
    ...

@staticmethod
def plot_classification_pie(
    results: list[ClassificationResult],
    save_path: str = None
) -> None:
    """
    ...

@staticmethod
def plot_expert_routing(
    X: np.ndarray,
    routes: np.ndarray,          # ID
    states: np.ndarray,
    save_path: str = None
) -> None:
    """
    ...

```

matplotlib, plotly, seaborn, sklearn.manifold (TSNE/PCA)

scx/utils/data_loader.py

```
class SCXDataLoader:
```

```
    """
```

```
    @staticmethod
```

```
    def make_synthetic(
```

```
        n_samples: int = 1000,
```

```
        n_states: int = 4,
```

```
        n_experts: int = 3,
```

```
        noise_ratio: float = 0.1,
```

```
        redundancy_ratio: float = 0.2,
```

```
        random_state: int = 42
```

```
) -> dict:
```

```
    """
```

```
    Returns
```

```
    -----
```

```
    dict with keys:
```

```
        X: (N, 2)
```

```
        y: (N,)
```

```
        states: (N,)
```

```
        experts: list of predict_fns
```

```
        expert_names: list of str
```

```
        phi: (N, d)
```

```
    """
```

```
    ...
```

```
    @staticmethod
```

```
    def make_multi_expert_classification(
```

```
        dataset: str = "cifar10",
```

```
        n_experts: int = 3,
```

```
        architectures: list[str] = None,
```

```
        noise_config: dict = None
```

```
) -> dict:
```

```
    """
```

```

        / label noise
        """
        ...

    @staticmethod
    def make_mlip_dataset(
        data_path: str = None,
        descriptor: str = "ace",
        n_configs: int = 1000
    ) -> dict:
        """ MLIP """
        ...

    @staticmethod
    def train_test_split_by_state(
        X: np.ndarray, y: np.ndarray,
        states: np.ndarray,
        test_ratio: float = 0.2,
        stratify: bool = True
    ) -> tuple:
        """/"""
        ...

```

numpy, sklearn.datasets, torchvision

scx/utils/evaluation.py

```

class SCXEvaluation:
    """SCX """

    @staticmethod
    def run_baseline_comparison(
        X: np.ndarray, y: np.ndarray,
        methods: list[str] = ["random", "uncertainty", "coreset", "scx"],
        budget: int = 100,
        n_trials: int = 5
    ) -> pd.DataFrame:
        """ baseline

```

```

Returns: DataFrame
"""

...

@staticmethod
def bootstrap_confidence(
    scores: np.ndarray,
    n_bootstrap: int = 1000,
    ci: float = 0.95
) -> tuple[float, float, float]:
    """Bootstrap """
    ...

@staticmethod
def effect_size(
    method_a: np.ndarray,
    method_b: np.ndarray
) -> float:
    """Cohen's d """
    ...

```

numpy, pandas, scipy.stats

1.3

```

SCXFramework
SCXConfig  ()
StateDiscovery  sklearn
    StateSpace  numpy/pandas
ExpertRegistry  ()
ExpertReliability  ExpertRegistry, StateAssignment
DataClassifier  numpy ()
StateValue  numpy
LearnabilityScore  sklearn/neighbors
NoiseScore  sklearn/neighbors
RedundancyScore  sklearn/neighbors

```

```

ExpertRouter  ExpertRegistry, ExpertReliability
ExpertConflict  numpy
ActionPolicy  DataClassifier, StateValue
    AcquisitionStrategy  numpy/sklearn
    CompressStrategy  numpy/sklearn
SCXMetrics  scipy/sklearn
Utils
    SCXVisualizer  matplotlib/plotly/sklearn
    SCXDataLoader  sklearn/torchvision
    SCXEvaluation  pandas/scipy

```

2.

Algorithm 1: State Discovery —

Algorithm: StateDiscovery

Input:

- X_{original} : (N, d_x)
- $X \rightarrow \hat{d}$
- method: "kmeans" | "gmm" | "spectral" | "hdbscan"
- n_{states} : K (HDBSCAN)
- optional hyperparameters (per method)

Output:

- StateSpace: {states: { $s_1, \dots, s_K$ }, s_i centroid, radius, count}
- hard_assignment: (N,) int
- soft_assignment: (N, K) float

Procedure:

- Step 1: Representation
 - = (X_{original}) # (N, d)
- Step 2: Dimensionality reduction (optional)
 - If $\text{dim}() > 50$:
 - $_{\text{proj}} = \text{PCA}(, n_{\text{components}}=\min(50, N-1))$
 - Else:

```
_proj =
```

Step 3: Clustering - method dispatch

Switch method:

```
case "kmeans":
```

```
    labels, centroids = KMeans(_proj, K, n_init=20)
    soft = softmax(-dist(_proj, centroids) / temperature)
```

```
case "gmm":
```

```
    gmm = GMM(_proj, K, covariance_type='full')
    labels = gmm.predict(_proj)
    soft = gmm.predict_proba(_proj)
    centroids = gmm.means_
```

```
case "spectral":
```

```
    adjacency = rbf_kernel(_proj, gamma=1/p)
    labels = SpectralClustering(adjacency, K)
    centroids = [mean(_proj[labels==k]) for k in range(K)]
    soft = softmax(-dist(_proj, centroids) / temperature)
```

```
case "hdbscan":
```

```
    clusterer = HDBSCAN(min_cluster_size=10)
    labels = clusterer.fit_predict(_proj)
    # -1 = noise,
    unique_labels = set(labels) - {-1}
    K_actual = len(unique_labels)
    centroids = [mean(_proj[labels==k]) for k in unique_labels]
    #
    soft = softmax(-dist(_proj, centroids) / temperature)
```

Step 4: Build StateSpace

For k = 0 to K-1:

```
    mask = (labels == k)
    state_info = StateInfo(
        id = k,
        centroid = centroids[k],
        radius = mean(dist(_proj[mask], centroids[k])),
        count = sum(mask),
```



```

        proportion = sum(mask) / N
    )
    state_space.add_state(state_info)

Step 5: Post-processing
    If method in {"kmeans", "spectral"}:
        #
        stability_scores = bootstrap_stability(_proj, method, K, n_iter=20)
        If mean(stability_scores) < 0.7:
            Warning("State structure unstable, consider reducing K or changing method")

    Return StateSpace, hard_assignment, soft_assignment

```

Algorithm 2: Expert Reliability Estimation (Supervised + Unsupervised)

Algorithm: ExpertReliabilityEstimation

Input:

- L: (N_L, d_x) labeled data
- y_L: (N_L,) labels
- U: (N_U, d_x) unlabeled data (optional, can be empty)
- experts: {f₁, ..., f_M}
- state_assignment_L: (N_L,) state IDs for labeled data
- state_assignment_U: (N_U,) state IDs for unlabeled data (or None)
- K: number of states
- : SCX threshold
- min_samples: minimum samples per state

Output:

- R: (M, K) estimated State-Conditioned Expert Risk
- SCX: (M, K) estimated SCX Reliability
- confidence: (M, K) estimation confidence

Procedure:

PHASE 1: Supervised Estimation (on labeled set L)

```

Initialize R = zeros(M, K), SCX = zeros(M, K)
Initialize count_s = zeros(K)

For each expert m = 1 to M:
    y_pred_m = f_m(L) # m

    For each state k = 0 to K-1:
        mask = (state_assignment_L == k)
        n_k = sum(mask)
        count_s[k] = n_k

        If n_k >= min_samples:
            # Risk:
            losses = loss_fn(y_pred_m[mask], y_L[mask])
            R[m, k] = mean(losses)

            # SCX:
            SCX[m, k] = mean(losses < )
        Else:
            R[m, k] = NaN # min_samples shrinkage
            SCX[m, k] = NaN

PHASE 1b: Shrinkage Estimation (for small states)

For each state k where count_s[k] < min_samples:
    # +
    global_R_m = mean(R[m, :]) # m
    n_k = count_s[k]

    # James-Stein style shrinkage
    = n_k / (n_k + min_samples) #
    R[m, k] = * R_raw[m, k] + (1-) * global_R_m
    SCX[m, k] = * SCX_raw[m, k] + (1-) * global_SCX_m

```

PHASE 2: Unsupervised Cross-Validation (on unlabeled set U, if available)

If U is not empty:

For each state $k = 0$ to $K-1$:

$\text{mask}_U = (\text{state_assignment}_U == k)$

$n_u = \text{sum}(\text{mask}_U)$

 If $n_u < \text{min_samples}$: continue

 #

$\text{preds} = \text{zeros}(M, n_u)$

 For $m = 1$ to M :

$\text{preds}[m, :] = f_m(U[\text{mask}_U])$

 # $D \in \mathbb{R}^{M \times M}$

$D = \text{zeros}(M, M)$

 For $i = 1$ to M :

 For $j = i+1$ to M :

$D[i, j] = D[j, i] = \text{mean}(\text{loss_fn}(\text{preds}[i], \text{preds}[j]))$

 #

$\text{mean_divergence} = \text{mean}(D, \text{axis}=1)$ #

$\text{outlier_experts} = \text{where}(\text{mean_divergence} > \text{mean}(\text{mean_divergence}) + 2 * \text{std}(\text{mean_divergence}))$

 # R:

 For m in outlier_experts :

$R[m, k] = \min(1.0, R[m, k] * 1.2)$ # 20%

$\text{confidence}[m, k] *= 0.8$

PHASE 3: Confidence Estimation

For each (m, k) :

$n = \text{count}_s[k]$

 If $n \geq \text{min_samples}$:

 # Bootstrap

```

        bootstrap_std = bootstrap(R[m, k], n_iter=100)
        confidence[m, k] = 1.0 / (1.0 + bootstrap_std)
    Else:
        confidence[m, k] = n / (n + min_samples) * 0.5    #

Return R, SCX, confidence

```

Algorithm 3: Data Four-Classification

Algorithm: DataFourClassification

Input:

- mean_errors: (K,) r(s) for each state
- proportions: (K,) (s) for each state
- learnabilities: (K,) L(s) for each state
- redundancies: (K,) D(s) for each state
- scx_matrix: (M, K) SCX_m(s) for all experts and states
- thresholds: {_r, _, _L, _D, _conflict}
- X_phi: (N, d) representation (optional, for sample-level refinement)
- y: (N,) labels (optional, for sample-level refinement)

Output:

- state_classifications: list of (state_id, category, confidence, reason)
- sample_noise_flags: (N,) bool (optional)

Procedure:

PHASE 1: State-Level Classification

For each state k = 0 to K-1:

```

    r = mean_errors[k]
    = proportions[k]
    L = learnabilities[k]
    D = redundancies[k]
    scx_range = max(scx_matrix[:,k]) - min(scx_matrix[:,k])
    max_scx = max(scx_matrix[:,k])

```

```

# Rule 1: Valuable -
If r > _r AND > _ AND L > _L AND D < _D:
    class = "valuable"
    conf = 0.9
    reason = f"r={r:.3f}>_r, ={: .3f}>_, L={L:.3f}>_L, D={D:.3f}<_D"

# Rule 2: Redundant -
Elif r < _r/2 AND > _ AND D > _D:
    class = "redundant"
    conf = 0.85
    reason = f"r={r:.3f}<_r/2, D={D:.3f}>_D"

# Rule 3: Expert-Dependent -
Elif scx_range > _conflict:
    class = "expert_dependent"
    conf = min(0.9, scx_range)
    reason = f"SCX range={scx_range:.3f}>_conflict, best={max_scx:.3f}"

# Rule 4: Noisy -
Elif r > _r AND < _ AND L < _L:
    class = "noisy"
    conf = 0.8
    reason = f"r={r:.3f}>_r, ={: .3f}<_, L={L:.3f}<_L"

# Default: Valuable
Else:
    class = "valuable"
    conf = 0.5
    reason = "conservative default"

```

Append (k, class, conf, reason) to state_classifications

PHASE 2: Sample-Level Refinement (optional)

If X_{phi} and y are provided:
 For each sample $i = 1$ to N :

```

s_i = state assignment of sample i
r_i = loss of current model on sample i
_i = proportion of state s_i

If _i < _r AND r_i > _r:      # +
    # k-NN
    neighbors = kNN(X_phi, X_phi[i], k=5)
    neighbor_labels = y[neighbors]
    majority_label = mode(neighbor_labels)
    If y[i] != majority_label:
        noise_flags[i] = "label_noise"
    Elif dist(X_phi[i], centroid[s_i]) > 3 * radius[s_i]:
        noise_flags[i] = "outlier_noise"
    Else:
        noise_flags[i] = "hard_sample"

```

PHASE 3: Summary

```

summary = {
    "n_valuable": count(class == "valuable"),
    "n_redundant": count(class == "redundant"),
    "n_noisy": count(class == "noisy"),
    "n_expert_dependent": count(class == "expert_dependent"),
    "valuable_samples": sum(proportions[class == "valuable"] * N),
    "redundant_samples": sum(proportions[class == "redundant"] * N),
    "noisy_samples": sum(proportions[class == "noisy"] * N),
    "expert_dependent_samples": sum(proportions[class == "expert_dependent"] * N),
}

```

Return state_classifications, noise_flags (optional), summary

Algorithm 4: State-Wise Acquisition

Algorithm: StateWiseAcquisition

Input:

- state_space: StateSpace with K states
- V: (K,) state values V(s)
- class: (K,) state categories
- budget: B (total samples to acquire)
- strategy: "proportional" | "threshold" | "hybrid"
- X_phi: (N, d) representation (for within-state selection)
- selection_strategy: "random" | "uncertainty" | "diverse" | "coreset"

Output:

- acquisition_plan: list of (state_id, n_samples, [sample_indices])

Procedure:

STEP 1: Filter to valuable states only

```
valuable_mask = [class[k] == "valuable" for k in range(K)]
K_v = sum(valuable_mask)
If K_v == 0:
    #
    B_s = [B // K] * K
    B_s[0:B % K] += 1
    #
    Return plan with B_s
```

STEP 2: Budget allocation across states

```
V_valuable = V[valuable_mask]
Switch strategy:
    case "proportional":
        # V(s)
        total_V = sum(V_valuable)
        for idx, k in enumerate(where(valuable_mask)[0]):
            B_s[k] = floor(B * V_valuable[idx] / total_V)
        # 1
        B_s[where(valuable_mask)[0]] = max(1, B_s)
```

```

case "threshold":
    # top 70%
    V_thresh = percentile(V_valuable, 70)
    top_mask = V_valuable >= V_thresh
    n_top = sum(top_mask)
    base = B // n_top
    for idx, k in enumerate(where(valuable_mask)[0]):
        If top_mask[idx]:
            B_s[k] = base
        Else:
            B_s[k] = 0

case "hybrid": # RECOMMENDED
    # 70% top 60% V(s)
    # 30%
    primary_budget = floor(B * 0.7)
    secondary_budget = B - primary_budget

    V_thresh = percentile(V_valuable, 40) # top 60%
    top_mask = V_valuable >= V_thresh
    top_indices = where(valuable_mask)[0][top_mask]
    bottom_indices = where(valuable_mask)[0][~top_mask]

    n_top = len(top_indices)
    n_bottom = len(bottom_indices)

    If n_top > 0:
        total_V_top = sum(V[top_indices])
        for k in top_indices:
            B_s[k] = max(1, floor(primary_budget * V[k] / total_V_top))

    If n_bottom > 0:
        per_state = secondary_budget // n_bottom
        for k in bottom_indices:
            B_s[k] = per_state

```

STEP 3: Within-state sample selection


```

For each state k with B_s[k] > 0:
    state_indices = where(state_assignment == k)[0]
    X_state = X_phi[state_indices]

    n_select = min(B_s[k], len(state_indices))

    Switch selection_strategy:
        case "random":
            selected = random_choice(state_indices, n_select, replace=False)

        case "uncertainty":
            #
            uncertainties = model_uncertainty(X_state)
            top_uncertain = argsort(uncertainties, descending=True)[:n_select]
            selected = state_indices[top_uncertain]

        case "diverse":          # RECOMMENDED
            # k-center
            selected_indices = kcenter_greedy(X_state, n_select)
            selected = state_indices[selected_indices]

        case "coreset":
            #
            densities = estimate_density(X_state)
            weights = 1.0 / (densities + )
            weights = weights / sum(weights)
            selected = weighted_sample(state_indices, weights, n_select, replace=False)

    acquisition_plan.append((k, n_select, selected))

Return acquisition_plan

```

STEP 4: Budget check

```

total_allocated = sum(plan.n_samples)
If total_allocated < B:
    # V(s)
    surplus = B - total_allocated
    Allocate surplus to top V(s) states

```

Algorithm 5: SCX-Compress — Weighted Coreset

Algorithm: SCXCompress

Input:

- D_s : $\{(x_i, y_i)\}_{i=1}^{N_s}$ dataset for one state s
- (D_s) : (N_s, d) representation
- redundancy_scores : $(N_s,)$ per-sample redundancy score D_i
- keep_ratio : $(0, 1)$ compression ratio
- method : "weighted_random" | "kcenter" | "herding"
- τ : temperature for weight softmax (default 1.0)

Output:

- C_s : compressed dataset $\{(x_j, y_j, w_j)\}_{j=1}^{n_{\text{keep}}}$

Procedure:

STEP 1: Determine keep count

```
n_keep = max(1, floor(keep_ratio * N_s))
```

STEP 2: Compute sample weights from redundancy

```

# → ()
# → ()
weights = softmax(-keep_ratio * redundancy_scores)
# : w_i = exp(-keep_ratio * D_i) / ∑_j exp(-keep_ratio * D_j)
# : →

```

STEP 3: Subset selection

Switch method:

```
case "weighted_random":    # FAST, DEFAULT
    C_s = weighted_sample(D_s, weights=weights,
                          size=n_keep, replace=False)

    # w_j

case "kcenter":            # BALANCED COVERAGE
    # k-center: n_keep
    #
    C_s = []
    remaining = D_s
    For t = 1 to n_keep:
        #
        farthest_score = zeros(len(remaining))
        For i, x_i in enumerate(remaining):
            min_dist_to_selected = min(dist(x_i, C_s)) if C_s else 0
            farthest_score[i] = min_dist_to_selected * weights[i]
        x_star = remaining[argmax(farthest_score)]
        C_s.append(x_star)
    #

case "herding":            # MEAN PRESERVATION
    = mean((D_s))          #
    C_s = []
    _selected = zeros(0, d) #

    For t = 1 to n_keep:
        remaining = D_s \ C_s
        scores = zeros(len(remaining))
        For i, x_i in enumerate(remaining):
            _i = (x_i)
            #
            _current = mean(_selected) if len(C_s) > 0 else 0
            scores[i] = < - _current, _i>
```

```

x_star = remaining[argmax(scores)]
C_s.append(x_star)
_selected = stack(_selected, (x_star))

```

STEP 4: Weight adjustment for selected subset

```

#
total_w = sum(w_j for (_, _, w_j) in C_s)
C_s = [(x, y, w / total_w * n_keep) for (x, y, w) in C_s]
# = n_keep

```

Return C_s

Algorithm 6: Expert Routing

Algorithm: ExpertRouting

Input:

- x: single sample to route
- experts: {f₁, ..., f_M}
- state_space: StateSpace with K states
- R: (M, K) estimated state-conditioned risks
- : temperature for weight softmax (default 1.0)
- mode: "hard" | "soft"
- _hard: confidence threshold for hard routing (default 0.7)

Output:

- $\hat{y}$: predicted label
- route_info: {
 - expert_weights: (M,) if soft mode, or
 - chosen_expert: int if hard mode,
 - dominant_state: int,
 - confidence: float

Procedure:

STEP 1: Compute state assignment for x

```
_x = (x) # Compute representation
distances = dist(_x, state_space.centroids) # (K,) distances to centroids
            = softmax(-distances / temperature) # (K,) soft assignment weights

dominant_state = argmax()
state_confidence = max()
```

STEP 2: Compute expert weights

```
# For each expert m, compute weighted risk
expert_scores = zeros(M)
For m = 1 to M:
    expert_scores[m] = _{k=1}^{K} [k] * R[m, k] # Expected risk for expert m

# Convert to weights: lower risk → higher weight
expert_weights = softmax(- * expert_scores) # (M,)
```

STEP 3: Route decision

```
Switch mode:
case "hard":
    If state_confidence >= _hard:
        # →
        m_star = argmin_m R[m, dominant_state]
         $\hat{y} = f_{\{m\_star\}}(x)$ 
        route_info = {
            "chosen_expert": m_star,
            "dominant_state": dominant_state,
            "state_confidence": state_confidence,
            "method": "hard_state_route"
```

```

    }
Else:
    # → fallback
     $\hat{y} = \sum_{m=1}^M \text{expert\_weights}[m] * f_m(x)$ 
    route_info = {
        "expert_weights": expert_weights,
        "method": "soft_fallback"
    }

case "soft":
                                                    # RECOMMENDED
    #
    predictions = [f_1(x), f_2(x), ..., f_M(x)]

    If task is classification:
        #
         $\hat{y} = \text{argmax}_c \sum_{m=1}^M \text{expert\_weights}[m] * 1[f_m(x) == c]$ 
    Elif task is regression:
        #
         $\hat{y} = \sum_{m=1}^M \text{expert\_weights}[m] * f_m(x)$ 

    route_info = {
        "expert_weights": expert_weights,
        "dominant_state": dominant_state,
        "dominant_expert": argmax(expert_weights),
        "method": "soft_weighted_ensemble"
    }

```

STEP 4: Optional - ExpertConflict handling

```

#
If max(expert_weights) / min(expert_weights) < 2:      #
    conflict_flag = True
    # →
    route_info["conflict_flag"] = True
    route_info["predictions"] = [f_1(x), ..., f_M(x)]

```

Return $\hat{y}$, route_info

3.

3.1

SCX 1. 2. R_m(s) 3. valuable / redundant / noisy / expert-dependent 4. point-wise baseline 5. SCX-Compress

3.2

3.2.1 2D 4

```
states_config = {
    "s0": { # -
        "centroid": [2, 2], "cov": [[0.8, 0.2], [0.2, 0.8]], "n": 300, "label_fn": "x",
    },
    "s1": { # -
        "centroid": [7, 7], "cov": [[1.0, -0.3], [-0.3, 1.0]], "n": 300, "label_fn": "x",
    },
    "s2": { # -
        "centroid": [7, 2], "cov": [[0.5, 0], [0, 0.5]], "n": 200, "label_fn": "x",
        "noise_ratio": 0.3 # 30%
    },
    "s3": { # -
        "centroid": [2, 7], "cov": [[0.3, 0.1], [0.1, 0.3]], "n": 100, "label_fn": "x",
    }
}
```

: 900 (300+300+200+100), 2D

3.2.2 3

		s0	s1	s2	s3
f_A	Logistic	92%	78%	85%	65%
	Re-				
	gres-				
	sion				
f_B	Random	85%	90%	70%	80%
	Forest				
	(depth=3)				
f_C	k-NN	88%	85%	75%	75%
	(k=15)				

3.2.3 20% ””

```
def inject_redundancy(X, y, ratio=0.2, noise_scale=0.05):
    n_redundant = int(len(X) * ratio)
    indices = np.random.choice(len(X), n_redundant, replace=False)
    X_dup = X[indices] + np.random.normal(0, noise_scale, (n_redundant, 2))
    return np.vstack([X, X_dup]), np.hstack([y, y[indices]])
```

3.2.4

```
def phi_synthetic(X):
    """2D → 6D: + + RBF """
    x, y = X[:, 0:1], X[:, 1:2]
    return np.hstack([
        X,                                # : x, y
        x**2, y**2,                        # : x², y²
        x * y,                             # : x*y
        np.exp(-(x**2 + y**2) / 10)        # RBF
    ])

```

3.3 Baseline

	Method	
Random		RND
Uncertainty	/ margin sampling	US

	Method	
Coreset	k-center coreset	CS
SCX-StateValue	V(s)	SCX-SV
SCX-FourClass		SCX-FC
SCX-Compress	coreset	SCX-CP

3.4

Test Accuracy	$\text{Acc}(f, X_{\text{test}}, y_{\text{test}})$
Budget Efficiency	AUC of learning curve
Routing Accuracy	$P(m^* = \arg \min R_m(s))$
V(s) Rank Correlation	$\text{Spearman}(V_{\text{est}}, V_{\text{true}})$
Classification F1	Macro F1 on four classes
Compression Retention	$\text{Acc}_{\text{compressed}} / \text{Acc}_{\text{full}}$
Noise Detection Precision	$\text{TP} / (\text{TP} + \text{FP})$

3.5

20 ±

:

- Exp 1.1: - V(s)
- Exp 1.2: (10%) -
- Exp 1.3: (20%) -
- Exp 1.4: + - pipeline
- Exp 1.5: -

3.6

Figure 1:

- (a) 2D = =
- (b) PCA =
- (c) vs

Figure 2:

- (a) $R_m(s)$ 3×4
- (b) $SCX_m(s)$ 3×4
- (c) vs

Figure 3:

- (a) 2D =
- (b)
- (c) /

Figure 4: $V(s)$

- (a) $V(s)$
- (b) V_{est} vs V_{true} + Spearman

Figure 5:

- (a) Budget vs Accuracy SCX vs Baselines
- (b)

Figure 6:

- (a) 0.1 1.0 -
- (b) SCX-Compress vs Random/Coreset

Figure 7:

- (a) $V(s)$
- (b)

4. ML

4.1

ML benchmarkCIFAR-10 / CIFAR-100 SCX 1. 2. 3. SCX 4. SCX-Compress

4.2

CIFAR-10	60,000	10
CIFAR-100	60,000	100

TinyImageNet () 110,000 200

- : mean/std per channel - : RandomCrop(32, padding=4) + RandomHorizontalFlip - : feature embedding

4.3

4.3.1 """

```
experts_config = {
    "ResNet18": {
        "model": ResNet18(num_classes=10),
        "train_config": {"epochs": 100, "lr": 0.1, "weight_decay": 5e-4},
        "strong_on": ["airplane", "ship", "truck"], #
    },
    "WideResNet-16-8": {
        "model": WideResNet(depth=16, widen_factor=8, num_classes=10),
        "train_config": {"epochs": 100, "lr": 0.1, "weight_decay": 5e-4},
        "strong_on": ["cat", "dog", "deer"], #
    },
    "ViT-Tiny": {
        "model": ViT(image_size=32, patch_size=4, num_classes=10),
        "train_config": {"epochs": 150, "lr": 0.01, "weight_decay": 0.1},
        "strong_on": ["bird", "frog", "horse"], #
    }
}
```

4.3.2 Proposition 1"""

1. CIFAR-10 3 : $G1=\{0,3,8\}$, $G2=\{2,5,7\}$, $G3=\{1,4,6,9\}$
2. Expert A: $G2$ 70%
3. Expert B: $G3$ 70%
4. Expert C: $G1$ 70%

: ""
→ SCX

4.3.3

```
def inject_class_conditional_noise(y, noise_ratio=0.2, symmetric=True):
    """
    y_noisy = y.copy()
    n_noise = int(len(y) * noise_ratio)
    noise_indices = np.random.choice(len(y), n_noise, replace=False)

    for idx in noise_indices:
        if symmetric:
            # :
            candidates = [c for c in range(n_classes) if c != y[idx]]
            y_noisy[idx] = np.random.choice(candidates)
        else:
            # : (e.g., catdog, truckautomobile)
            y_noisy[idx] = asymmetric_flip(y[idx])
    return y_noisy, noise_indices
```

4.3.4

1. 15%
2. (brightness $\pm 10\%$, contrast $\pm 10\%$)
3. ""

: SCX-RedundancyScore

4.4

```
# ResNet18 penultimate layer
phi_fn = lambda x: resnet18_feature(x)    # (N, 512)

#
state_discovery = StateDiscovery(method="kmeans", n_states=20)
# 20CIFAR-10 10 1-3 easy/hard/mixed

#
state_class_confusion = confusion_matrix(true_labels, state_assignment)
# :
```

4.5

H1: SCX >

Experiment 2.1: Reliability Estimation

1. 3
2. SCX R_m(s) SCX_m(s)
3. :
 - vs
 - ""vs"" R_m(s)

Expected: 10-20%

H2: SCX > uncertainty

Experiment 2.2: Active Learning Comparison

1. : 10 (CIFAR-10: 100)
2. 100 , 10 (budget = 1000)
3. Baseline:
 - Random
 - Uncertainty (margin sampling)
 - Coreset (k-center)
 - SCX-StateValue (V(s))
 - SCX-FourClass ()
- 4.

Expected: SCX (budget < 500)

H3: SCX

Experiment 2.3: Noise Detection

1. label noise (10%, 20%, 30%)
2. SCX NoiseScore DataClassifier
3. : Precision, Recall, F1 on noise detection

Expected: SCX F1 > 0.7 ()

H4: SCX-Compress

Experiment 2.4: Data Compression

1. CIFAR-10 (50,000)
2. SCX-Compress 50%, 25%, 10%
3. ResNet18
4. : Random , Coreset

Expected: SCX-Compress 50% 97%+

H5:

Experiment 2.5: Ablation

1. V(s) :
 - remove r(s) $\rightarrow$ density * learnability
 - remove (s) $\rightarrow$ error * learnability
 - remove L(s) $\rightarrow$ error * density
 - remove D(s) $\rightarrow$
2. :
 - KMeans vs GMM vs Spectral vs HDBSCAN
3. K :
 - K = 5, 10, 20, 30, 50

Expected: L(s) D(s)

4.6 Baseline

	#	Baseline	
1	Random		sklearn
2	Uncertainty (Margin)	margin	modAL /
3	Uncertainty (Entropy)		modAL /
4	BALD	Bayesian AL by Disagreement	baal
5	Coreset (k-center)		
6	Coreset (KCenterGreedy)	k-center	
7	Learning Loss	(Yoo & Kweon 2019)	
8	VAAL	Variational Adversarial AL	
9	SCX-StateValue (Ours)	V(s)	SCX
10	SCX-FourClass (Ours)		SCX

#1-#5 #9-#10 #7-#8

4.7

: 1×GPU (RTX 3060 / 4090)
2-4 (CIFAR-10), 4-8 (CIFAR-100)

:

Exp					Budget				
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----
2.1	C10		0%	0%	3	20	-	5	
2.2	C10		10%	15%	3	20	1000	20	
2.3	C10		20%	15%	3	20	1000	20	
2.4	C100		10%	15%	3	50	2000	20	
2.5	C10		20%	15%	3	20	500	20	

5. MLIP

5.1

MLIPMachine Learning Interatomic Potentials SCX 1. 2. MLIPACE vs NEP
vs MACE 3. SCX DFT 4. EGP Paper 1 AlGa_N gauge-fixing

5.2

5.2.1 EGP Paper 1 AlGa_N - wurtzite AlGa_N Al , 0~100% - ~2000-5000 -
DFT (eV) + (eV/Angstrom) - ~72-128 /

5.2.2

```
def compute_ace_descriptors(structures, max_degree=4, cutoff=6.0):  
    """ ACE (Atomic Cluster Expansion)  
  
    Parameters  
    -----  
    structures : list[ase.Atoms]  
    max_degree : int  
        ACE  
    cutoff : float
```

(Angstrom)

Returns

X_phi : *np.ndarray*, *shape* (*n_atoms*, *d*)

ACE

structure_ids : *np.ndarray*, *shape* (*n_atoms*,)

structure index

"""

pyace pacemaker ACE

~200-500

descriptors = []

for atoms in structures:

 # *ACE : 2-body, 3-body /*

 desc = pyace.calculate_descriptors(atoms, max_degree, cutoff)

 descriptors.append(desc)

return np.vstack(descriptors)

||||| |-----|-----|-----| | ACE (Basis) | 200-500 | || | SOAP | 200-1000
|||| | MBTR | 100-500 | || | Atom-centered symmetry functions | 50-200 |
Behler ||

5.3

s_Al-bulk	Al	Al 12	ACE:
s_Ga-bulk	Ga	Ga 12	ACE:
s_Al-Ga-mix	Al-Ga	Al Ga	ACE:
s_surface		< 12	ACE:
s_defect		/	ACE:
s_N-rich	N	N	ACE: N
s_strained		±10%+	ACE:

5.3.1

A:

state_discovery = StateDiscovery(method="hdbscan", n_states="auto")

HDBSCAN outlier state


```

# B:
kmeans = KMeans(n_clusters=7, init=physical_centroids)
#

# C:
# →
# Level 1: (bulk vs surface vs defect)
# Level 2: (Al-rich vs Ga-rich vs mixed)

```

5.4

```

experts = {
    "ACE": {
        "model": pacemaker.ACE(
            max_degree=4, cutoff=6.0,
            n_hidden=20, n_basis=8
        ),
        "description": "Atomic Cluster Expansion - polynomial basis",
        "training": "all AlGaN structures",
        "strength": "overall accuracy, good transferability",
    },
    "NEP": {
        "model": gpumd.NEP(
            version=4, cutoff=6.0,
            num_parameters=4000
        ),
        "description": "Neuroevolution Potential - neural network",
        "training": "all AlGaN structures",
        "strength": "fast, good for disordered systems",
    },
    "MACE": {
        "model": mace.MACE(
            r_max=6.0, num_interactions=3,
            hidden_irreps="128x0e + 128x1o"
        ),
        "description": "MACE - equivariant message passing",
        "training": "all AlGaN structures",
    },
}

```

```

    "strength": "state-of-the-art accuracy, data hungry",
  }
}

```

5.5

H1: MLIP

Experiment 3.1: Expert Reliability Map

1. AlGaN ACE
- 2.
3. s $R_m(s)$ (energy MAE + force MAE)
4. $R_m(s)$ (3 experts $\times$ K states)

Expected:

- ACE bulk ()
- NEP disordered/mixed
- MACE surface/defect ()
- Proposition 1:

H2: SCX

Experiment 3.2: Active Learning for DFT

Setting: DFT

1. : 100
2. : 50 , DFT ,
3. 10 (budget = 500)
4. Baseline:
 - Random:
 - Uncertainty: ensemble
 - SCX: ($V(s)$)
 - SCX-Diverse: +
5. energy/force MAE

Expected: SCX 300-500 800-1000

H3: SCX

Experiment 3.3: Data Cleaning

1. :
 - 10% (noisy)
 - 10% (redundant)
2. SCX
3. :
 - NoiseScore
 - RedundancyScore
 - noisy/redundant

Expected: SCX-identified noisy data MAE 5-10%

H4:

Experiment 3.4: Expert Routing

Setting: SCX expert

1. $R_m(s)$
2. :
 - a. : $R_m(s)$ expert
 - b. : $R_m(s)$ ensemble
 - c. : expert
3. energy/force MAE

Expected:

H5: EGP Paper 1

Experiment 3.5: Gauge Fixing Cross-Validation

Setting: SCX EGP gauge-fixing

1. EGP Paper 1 "gauge" :
 - expert
 - Gauge fixing expert
2. SCX gauge fixing :
 - : $R_m(s)$
 - : $R_m'(s)$ -
3. : gauge fixing SCX

Expected: SCX gauge fixing

5.6

Energy MAE	mean(	E_pred - E_DFT
Force MAE	mean(	F_pred - F_DFT
Per-state RMSE	RMSE on each state	
SCX-Routing Gain	(MAE_global - MAE_routed) / MAE_global	
Acquisition Efficiency	AUC of learning curve	
Compression Retention	MAE_compressed / MAE_full	

6.

6.1 Python

```
# ===== () =====
numpy>=1.24.0          #
scipy>=1.10.0          #
scikit-learn>=1.3.0    #
pandas>=2.0.0          #
pyyaml>=6.0            #
typing-extensions>=4.0.0 #

# ===== () =====
matplotlib>=3.7.0      #
seaborn>=0.12.0        #
plotly>=5.15.0         #
scienceplots>=2.0.0    #

# ===== (ML benchmark) =====
torch>=2.0.0           # PyTorch
torchvision>=0.15.0    #
timm>=0.9.0            # PyTorch Image Models (ViT )
```

```

# ===== (Baseline) =====
modAL>=0.4.0          # Active Learning
baal>=1.1.0           # Bayesian Active Learning

# ===== MLIP () =====
pyace>=0.5.0          # ACE
mace>=0.3.0           # MACE
gpumd>=4.0            # NEP/GPUMD
ase>=3.22.0           #
matgl>=0.3.0          #

# =====
pytest>=7.4.0          #
pytest-cov>=4.1.0      #
black>=23.0.0          #
isort>=5.12.0          # import
mypy>=1.5.0            #
pre-commit>=3.3.0      #

```

6.2

```

# environment.yml
name: scx
channels:
  - pytorch
  - conda-forge
  - defaults
dependencies:
  - python=3.11
  - numpy>=1.24.0
  - scipy>=1.10.0
  - scikit-learn>=1.3.0
  - pandas>=2.0.0
  - pyyaml>=6.0
  - matplotlib>=3.7.0
  - seaborn>=0.12.0
  - plotly>=5.15.0
  - pip

```

```

- pip:
  - modAL>=0.4.0
  - scienceplots>=2.0.0

# pip
pip install numpy scipy scikit-learn pandas pyyaml matplotlib seaborn plotly
pip install modAL scienceplots

# ML benchmark
pip install torch torchvision timm

# MLIP
pip install ase pyace mace gpumd

```

6.3

	CPU					
	()					
Exp 1.1	1 core	2 GB	2 min	20	40 min	
Exp 1.2	1 core	2 GB	3 min	20	60 min	
Exp 1.3	1 core	2 GB	3 min	20	60 min	
Exp 1.4	1 core	2 GB	5 min	20	100 min	
Exp 1.5	1 core	2 GB	3 min	20	60 min	
						~5

ML Benchmark

	GPU					
	()					
C10 (3 models)	1×RTX3060	8 GB	2 h	1	2 h	
Exp 2.1 (Reliability)	1×RTX3060	8 GB	1 h	5	5 h	
Exp 2.2 (AL, 4 methods)	1×RTX3060	8 GB	3 h	20	60 h	
Exp 2.3 (Noise)	1×RTX3060	8 GB	3 h	20	60 h	
Exp 2.4 (Compress)	1×RTX3060	8 GB	1 h	20	20 h	
Exp 2.5 (Ablation)	1×RTX3060	8 GB	2 h	20	40 h	
ML						~190 h (8)

ML 1. CIFAR-10 3 × 4 methods 2. 3. ML

MLIP

	CPU	GPU		
ACE	8 cores	—	16 GB	1 h
(3 models)	8 cores	1×RTX4090	32 GB	24 h
Exp 3.1 (Reliability)	4 cores	—	16 GB	2 h
Exp 3.2 (AL)	8 cores	1×RTX4090	32 GB	48 h
Exp 3.3 (Cleaning)	4 cores	1×RTX4090	16 GB	8 h
Exp 3.4 (Routing)	4 cores	1×RTX4090	16 GB	4 h
MLIP				~87 h

6.4

Phase 1: (1-2)

```
scx/state/      -
scx/expert/     -
scx/valuation/  -
scx/action/     -
scx/core/       -
scx/utils/      -
tests/          -
```

Phase 2: (1)

```
experiments/synthetic/
Exp 1.1-1.5
```

Figure 1-7

Phase 3: ML Benchmark (2-3)

```
experiments/ml_benchmarks/
CIFAR-10
Exp 2.1-2.5
Figure 8-12
```

Phase 4: MLIP Case (1-2)

```
experiments/mlip_case/
ACE
```

Exp 3.1-3.5
Figure 13-17

Phase 5: (2 Phase 3/4)
paper/paper_text/
(Definitions + Propositions)
(Algorithm)
(Results)
arXiv

6.5

```
#  
python experiments/synthetic/run_all.py \  
    --output_dir outputs/synthetic \  
    --n_runs 20 \  
    --seed 42  
  
# ML Benchmark  
python experiments/ml_benchmarks/run_al.py \  
    --dataset cifar10 \  
    --method scx_state_value \  
    --noise_ratio 0.1 \  
    --budget 1000 \  
    --gpu 0  
  
# MLIP  
python experiments/mlip_case/run_routing.py \  
    --data_path /path/to/algan/structures.xyz \  
    --descriptor ace \  
    --output_dir outputs/mlip_case
```

X		—
Y		—
S		K
		$X \rightarrow \hat{d}$
$_s(x)$	$x \ s$	$(K,)$
f_m	m	$X \rightarrow Y$
M		—
K		—
$R_m(s)$	$m \ s$	(M, K)
$SCX_m(s)$	$m \ s$	(M, K)
$V(s)$	s	$(K,)$
$r(s)$	s	$(K,)$
(s)	s	$(K,)$
$L(s)$	s	$(K,)$
$D(s)$	s	$(K,)$
$C(s)$	s	$(K,)$
$N(x)$	x	$(N,)$
	SCX	
	softmax	